

4.3 MODELAGEM UML

4.3.1 Diagrama de casos de uso

O diagrama delimita as funcionalidades observáveis do sistema experimental e suas interações externas. A aplicação cliente é o ator primário, pois inicia os dois modos de registro. Apache Kafka e PostgreSQL são atores secundários de suporte: o primeiro intermedeia a entrega assíncrona e o segundo mantém o efeito persistente. Esses elementos aparecem fora da fronteira do protótipo para evidenciar dependências, sem tratá-los como objetivos do usuário.

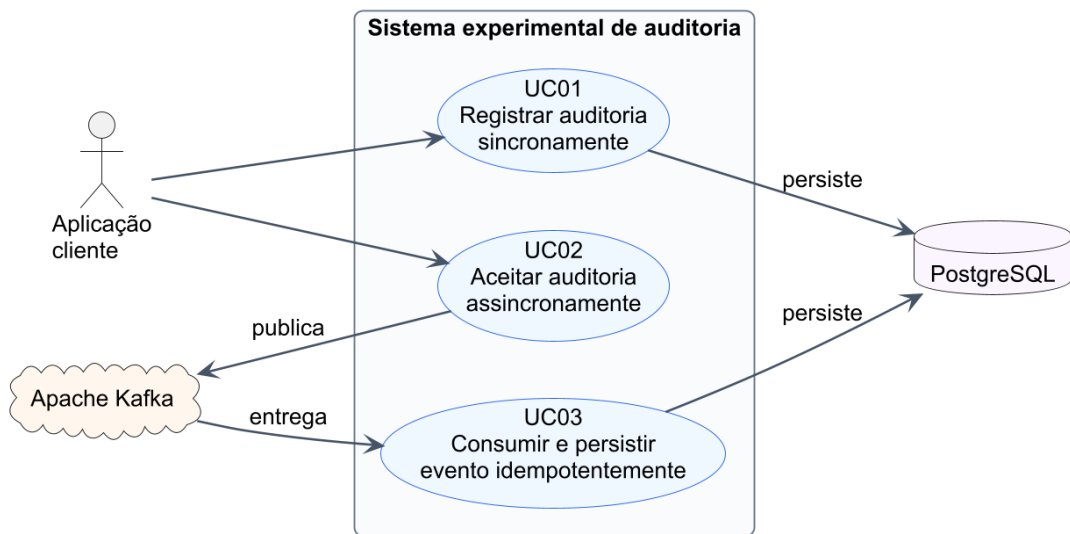


FIGURA 2 – Diagrama de casos de uso

FONTE: Elaborado pelo autor (2026).

4.3.1.1 Atores

- AT01 **Aplicação cliente**: sistema ou gerador de carga que envia um registro de auditoria válido para uma das variantes;
- AT02 **Apache Kafka**: infraestrutura que recebe o evento publicado e o disponibiliza posteriormente ao consumidor;
- AT03 **PostgreSQL**: sistema de persistência que mantém o registro e garante unicidade do UUID.

4.3.1.2 UC01 — Registrar auditoria sincronamente

Objetivo: receber, validar, processar e persistir um registro antes de devolver o controle à aplicação cliente. **Ator primário:** aplicação cliente. **Pré-condições:** API síncrona e PostgreSQL disponíveis; evento montado conforme o contrato. **Ativação:** envio de `POST /audit` à API síncrona. **Pós-condição de sucesso:** registro persistido ou reentrega idêntica reconhecida sem criação de uma segunda linha.

No fluxo principal, a aplicação envia o evento; a API valida os campos; calcula a representação canônica e o SHA-256; solicita a persistência idempotente; e responde 201 **Created** com o UUID, o status e o hash. Como fluxo alternativo, quando o UUID e o hash já existem, a operação retorna o status **duplicate**. Como exceções, contrato inválido produz 422 **Unprocessable Entity**, enquanto o mesmo UUID associado a hash diferente caracteriza conflito de integridade.

4.3.1.3 UC02 — Aceitar auditoria assincronamente

Objetivo: validar o evento e aceitá-lo para processamento posterior, sem aguardar a persistência no PostgreSQL. **Ator primário:** aplicação cliente. **Ator secundário:** Apache Kafka. **Pré-condições:** API produtora inicializada e produtor Kafka disponível. **Ativação:** envio de POST /audit à API assíncrona. **Pós-condição de sucesso:** evento aceito pela API produtora e identificado por UUID; a persistência ainda não é considerada concluída.

No fluxo principal, a aplicação envia o evento; a API valida o contrato; encaminha a mensagem ao produtor Kafka; e responde 202 **Accepted**. Contrato inválido produz 422 **Unprocessable Entity**. Produtor indisponível ou buffer sem capacidade produz 503 **Service Unavailable**. A confirmação 202 representa somente aceitação, conforme a RN06.

4.3.1.4 UC03 — Consumir e persistir evento idempotentemente

Objetivo: concluir o processamento iniciado pela variante assíncrona e impedir efeitos duplicados. **Ator de ativação:** Apache Kafka. **Ator secundário:** PostgreSQL. **Pré-condições:** consumidor ativo, tópico configurado e mensagem disponível. **Ativação:** entrega de uma mensagem ao consumidor. **Pós-condição de sucesso:** evento persistido ou reconhecido como reentrega idêntica, seguido da confirmação do offset.

No fluxo principal, o consumidor recebe a mensagem; reconstrói e valida o evento; calcula o hash; persiste o registro; e confirma o offset. Quando UUID e hash já existem, a persistência é tratada como idempotente. Se a persistência falhar, o offset não deve ser confirmado, permitindo nova entrega. O mesmo UUID com hash diferente caracteriza conflito de integridade e deve ser registrado para diagnóstico.

4.3.2 Diagrama de atividades

As Figuras 3 e 4 detalham as decisões e os caminhos de exceção dos dois fluxos principais. O primeiro fluxo possui uma única unidade temporal: a resposta somente ocorre depois da tentativa de persistência. O segundo separa a aceitação HTTP do processamento posterior, deixando explícito que 202 **Accepted** não comprova a existência da linha no PostgreSQL.

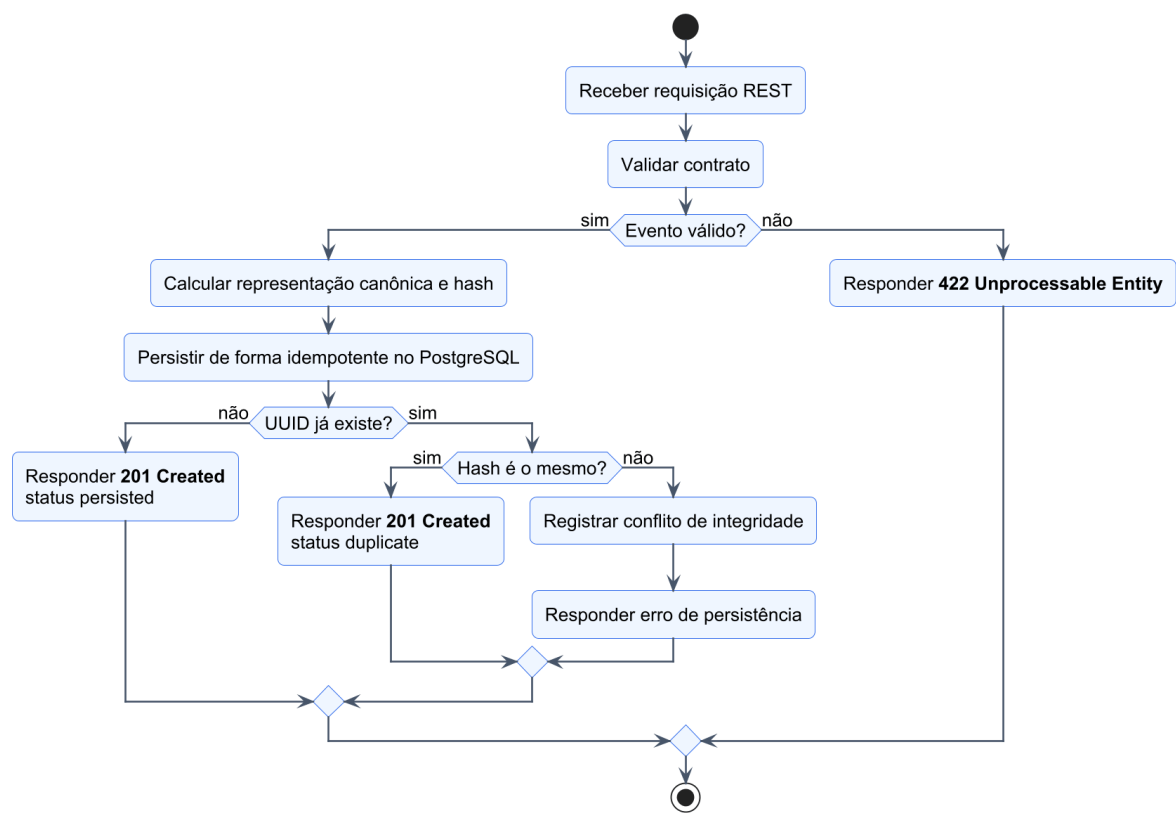


FIGURA 3 – Atividade de processamento síncrono

FONTE: Elaborado pelo autor (2026).

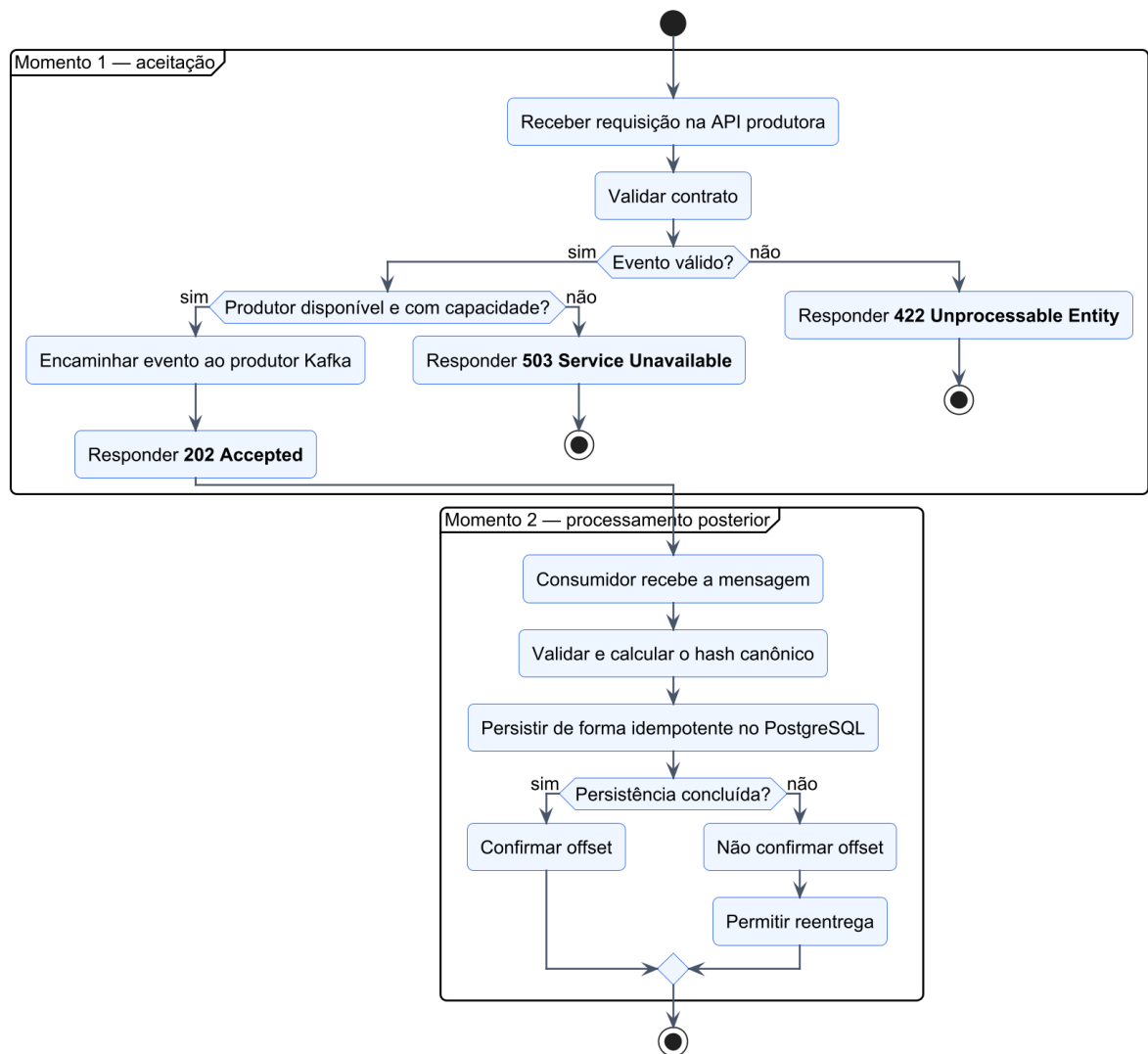


FIGURA 4 – Atividade de processamento assíncrono

FONTE: Elaborado pelo autor (2026).